

BASIC Practical Task

Practise Task for array

Task: Building a Recipe Management System

Brief Description: You are tasked with building a Recipe Management System using JavaScript. This system will allow users to manage recipes by adding, editing, deleting, and listing recipes. It will include features for categorizing recipes, searching by ingredients, and saving recipes to local storage. Asynchronous operations will simulate fetching and saving data, and error handling will be implemented to manage potential issues gracefully.

Practise Task for String

Task: Implementing a Text Analysis Tool

Brief Description: You are tasked with building a JavaScript tool that analyzes text input to provide various insights such as character count, word count, and frequency of each word. This tool will utilize functions for processing text, arrays/objects for storing and manipulating data, string methods for parsing and analyzing text, and variables to manage input and output.

Practical Task: Simple Task Manager Application

Goal

Develop a Simple Task Manager application using JavaScript that allows users to add, view, update, and delete tasks. This project will help you practice and understand the usage of various JavaScript array methods in a practical context.

Description

The Simple Task Manager application will consist of a user interface that allows users to manage their tasks efficiently. Each task will have properties such as `id`, `description`, and `completed` status. The application will utilize various JavaScript array methods to perform operations on the list of tasks.

Features

1. Add New Task

- Users can add new tasks to the list by providing a description.
- Each new task will be appended to the task list array.

2. View Tasks

- Display the list of tasks, showing their description and completion status.
- Users can see both completed and incomplete tasks.

3. Update Task

- Users can update the description of existing tasks.
- Toggle the completion status of tasks (mark as complete/incomplete).

4. Delete Task

- Users can remove tasks from the list.

5. Filter Tasks

- Users can filter tasks to view only completed or only incomplete tasks.

PRACTICAL TASK :-

1 . Task: Form Validation

Objective:

Create a form with dynamic validation that provides real-time feedback to users as they fill out the form. The goal is to enhance user experience by ensuring that form inputs meet specified criteria before submission.

Requirements:

1. User Interface:

- **Form Fields:**
 - Name
 - Email
 - Password
 - Confirm Password
- **Validation Messages:**
 - Real-time feedback messages for each field that indicate validation status.
- **Submit Button:**
 - A button to submit the form (initially disabled until all fields are valid).

2. Validation Criteria:

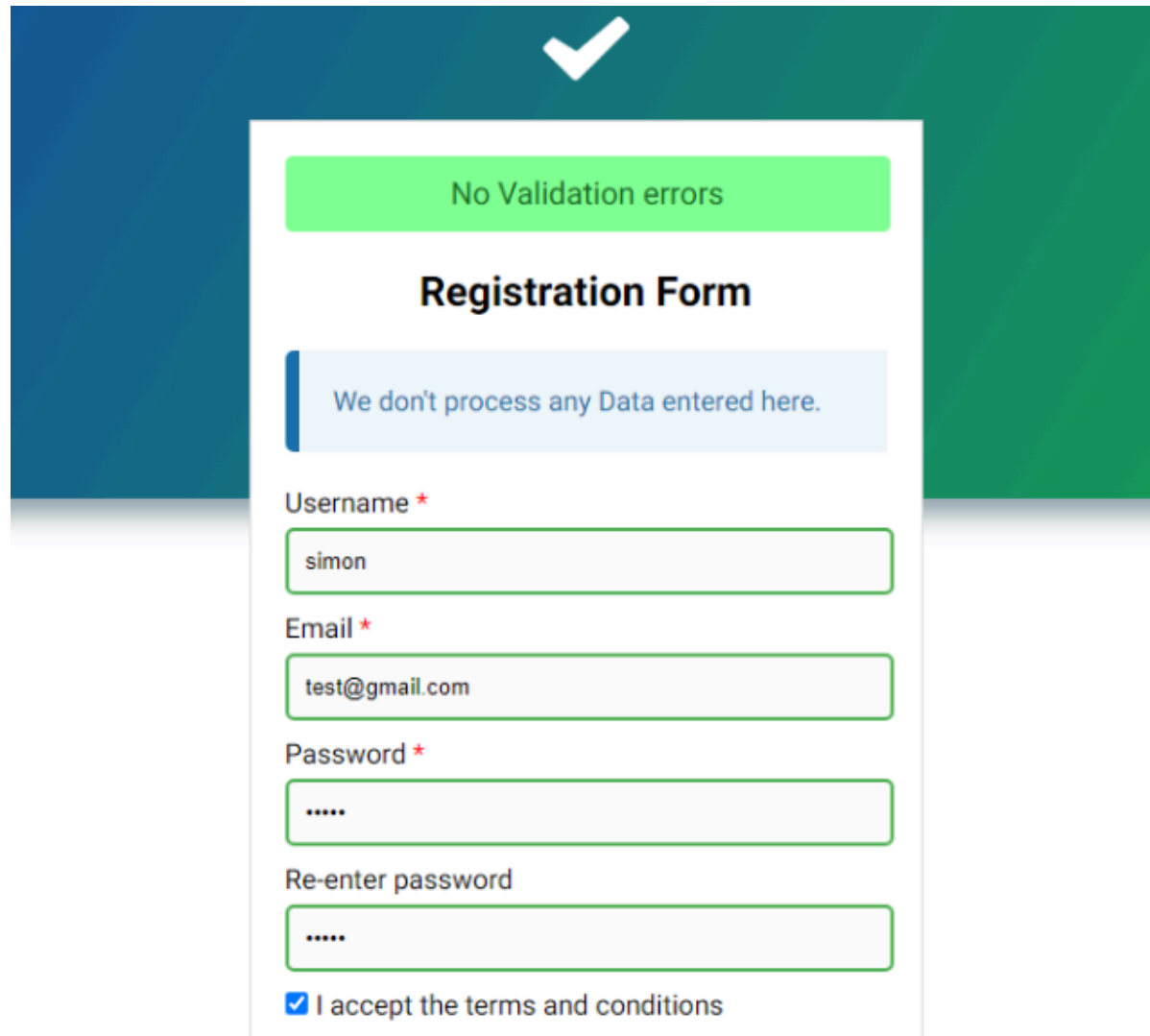
- **Name:**
 - Must be at least 3 characters long.
 - Must not contain numbers or special characters.
- **Email:**
 - Must be in a valid email format (e.g., user@example.com).
- **Password:**
 - Must be at least 8 characters long.
 - Must include at least one uppercase letter, one lowercase letter, one number, and one special character.
- **Confirm Password:**
 - Must match the password field.

3. Functionality:

- **Real-Time Validation:**
 - Validate each field as the user types.
 - Display appropriate error messages when a field is invalid.

- Display a success message when a field is valid.
- **Form Submission:**
 - Enable the submit button only when all fields are valid.
 - Prevent form submission if any field is invalid.
 - Display a summary message indicating whether the form was submitted

- successfully or which fields are still invalid.



A registration form is displayed on a dark teal background. At the top, a large white checkmark is centered. Below it, a light green box contains the text "No Validation errors". The form title "Registration Form" is centered in bold. A light blue box with a dark blue border on the left contains the text "We don't process any Data entered here.". The form fields are: "Username *" with the value "simon", "Email *" with the value "test@gmail.com", "Password *" with masked characters "*****", and "Re-enter password" with masked characters "*****". At the bottom, there is a checked checkbox and the text "I accept the terms and conditions".

✓

No Validation errors

Registration Form

We don't process any Data entered here.

Username *

simon

Email *

test@gmail.com

Password *

Re-enter password

☒ I accept the terms and conditions

2 . Task : To-Do List Application

Objective:

Create a simple To-Do list application that allows users to:

1. Add new tasks.
2. Mark tasks as completed.
3. Delete tasks

Requirements:

1. User Interface:

- A text input for adding new tasks.
- A button to add the task.
- A list to display tasks.
- Each task should have a checkbox to mark it as completed.
- Each task should have a delete button to remove it.

2. Functionality:

- Tasks should be added to the list with the input text.
- When a task is marked as completed, it should be visually distinct (e.g., strikethrough text).
- Tasks can be deleted from the list.

TODO App

Add New Task



Tasks

study

2022-04-20

Learn JavaScript



workout

2022-04-28

Go to gym



3 .Task: Display and Search a List of Users in a Table Using External API

Objective:

Create a JavaScript application that fetches a list of users from an external API, displays the users in a table, and allows the user to search through the list by name.

Taken a API from Here - <https://jsonplaceholder.typicode.com/>

Requirements:

1. User Interface:
 - A search input field to filter users by name.
 - A button to trigger the search.
 - A table to display the list of users with columns for Name, Email, and Address.
2. Functionality:
 - Fetch the list of users from an external API (e.g., JSONPlaceholder).
 - Display the fetched users in the table.
 - Filter the displayed users based on the search input.

Search Table Record:

Item to filter..

Name	Email	Phone	ID
Asif Mughal	admin@codehim.com	0123456789	13213
John Doe	hello@codehim.com	9876543210	34319
Maria Khan	maria@example.com	6754328901	13299
Zeshan Ali	zee@example.com	6754328901	123456
John Dev	john@example.com	6754328901	42432

4 . Extra Task

1. **Quiz Application Task:** Create a quiz application that allows users to answer multiple-choice questions.

Requirements:

- **User Interface:**
 - A question displayed with multiple-choice answers.
 - A button to submit the answer.

- A display of the user's current score and question number.
- **Functionality:**
 - Present questions one at a time.
 - Move to the next question after an answer is submitted.
 - Display the final score after the last question.

2. Calculator Application *Task:* Create a basic calculator application that can perform simple arithmetic operations.

Requirements:

- **User Interface:**
 - Input fields for entering two numbers.
 - Buttons for addition, subtraction, multiplication, and division.
 - A display area for showing the result.
- **Functionality:**
 - Perform the selected arithmetic operation when the corresponding button is clicked.
 - Display the result in the result area.
 - Validate input to ensure numbers are entered

3. Quote of the Day Application *Task:* Create an application that fetches and displays a random quote from an API.

Requirements:

- **User Interface:**
 - A button to fetch a new quote.
 - An area to display the quote and the author.
- **Functionality:**

- Fetch a random quote from a quotes API using Promises or async/await.
- Display the fetched quote in the designated area.
- Handle errors if the API request fails.

4. Image Carousel Task: Create an image carousel that cycles through a set of images.

Requirements:

- **User Interface:**
 - A container to display images.
 - Buttons to navigate to the previous and next images.
- **Functionality:**
 - Display images one at a time.
 - Cycle through images using the previous and next buttons.
 - Implement automatic cycling through images every few seconds.

5. Guess the Number Game Task: Create a game where the user has to guess a randomly generated number within a certain range.

Requirements:

- **User Interface:**
 - An input field to enter the guess.
 - A button to submit the guess.
 - A display area to show hints (e.g., "Too high", "Too low") and the number of attempts.
- **Functionality:**
 - Generate a random number between 1 and 100.
 - Check if the guess is correct and provide hints.

- Track the number of attempts and display it.
- Allow the user to play again after guessing correctly.

6. Countdown Timer

- **Task:** Create a countdown timer that counts down from a given number of seconds.
- **Requirements:**
 - Use `setInterval` to decrement the countdown value every second.
 - Display the remaining time on the webpage.
 - Stop the countdown when it reaches zero and display a message.

7. Multiple API Requests

- **Task:** Create a script that fetches data from multiple APIs and combines the results.
- **Requirements:**
 - Use the Fetch API to make multiple requests.
 - Use promises to handle the asynchronous operations.
 - Combine the results from different APIs and display them on the webpage.

8. Dynamic Form Builder

- **Task:** Create a script that dynamically builds and appends form elements based on user input.
- **Requirements:**
 - Use `prompt` to get the number and type of form elements (e.g., text input, checkbox).
 - Dynamically create and append the form elements to the DOM.
 - Display the form on the webpage.

9. Task: Interactive Photo Gallery

Task: Create an interactive photo gallery that allows users to view, add, and delete photos.

Requirements:

- a. Display a grid of photos using DOM manipulation.
- b. Add functionality to view a larger version of a photo in a modal when clicked.
- c. Allow users to add new photos by providing a URL.
- d. Implement a delete button for each photo to remove it from the gallery.

10. To-Do List with Drag-and-Drop

- **Task:** Create a to-do list application with drag-and-drop functionality to reorder tasks.
- **Requirements:**
 - Allow the user to add new tasks.
 - Implement drag-and-drop to reorder tasks.
 - Update the task order in the DOM based on user actions.

11. Data Analysis Dashboard

- **Task:** Develop a data analysis dashboard that processes and displays statistics from a dataset using loops.
- **Requirements:**
 - Use loops to calculate and display metrics such as average, sum, minimum, and maximum values.
 - Visualize data using charts (e.g., bar chart, line chart) generated dynamically from the dataset.
 - Allow users to filter and interact with the data for deeper analysis.

12. Text Analysis Tool

- **Task:** Develop a tool that analyzes text input to calculate word frequency, average word length, and most common words.
- **Requirements:**
 - Use array methods to process and manipulate text (e.g., split, map, reduce).
 - Display statistics such as word count, average word length, and list of common words.
 - Handle punctuation, case sensitivity, and special characters appropriately.

13. Music Player with Playlist

- **Task:** Build a music player application that plays songs asynchronously and manages playlists.
- **Requirements:**
 - Use callbacks to handle song playback events (play, pause, stop).
 - Implement features such as adding/removing songs from playlists and shuffling songs.
 - Display current song information and playlist dynamically.